

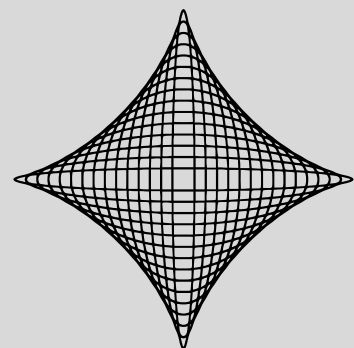


# **Spatial Entity Management**

---

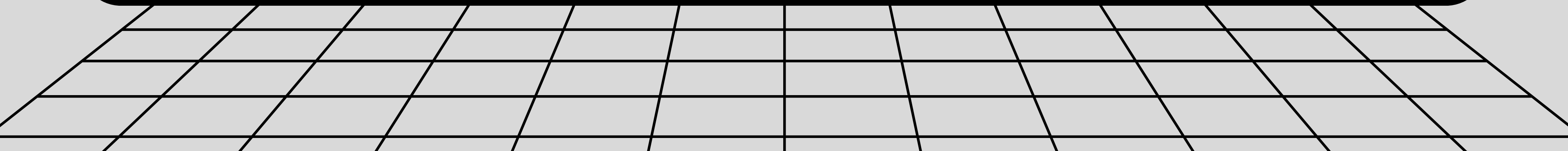
**By**

|                              |                   |
|------------------------------|-------------------|
| <b>Alvyn Ilham Kurniawan</b> | <b>2902696281</b> |
| <b>Josh Lailo Soli Daeli</b> | <b>2902704983</b> |
| <b>Rangga Wijaya</b>         | <b>2902671865</b> |



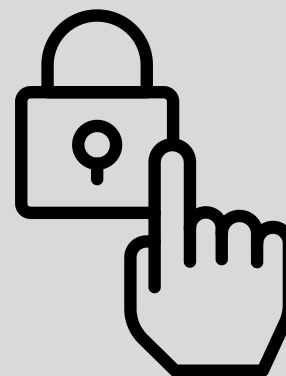
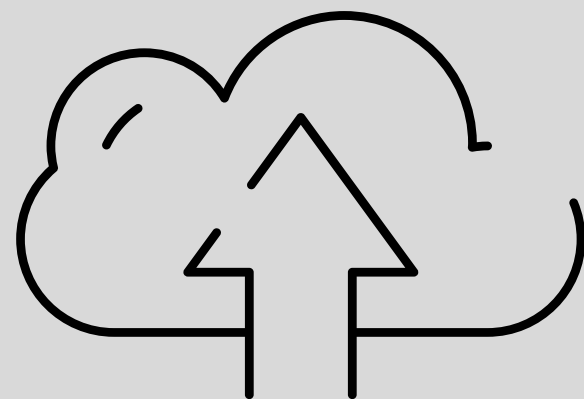
# Background

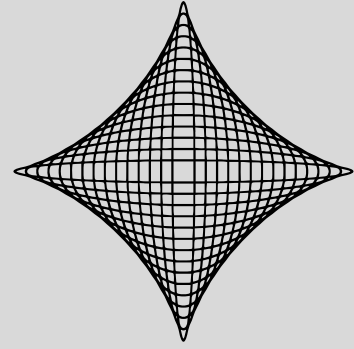
**Simulating 2D environments requires coordinates to represent entity object positions. A spatial environment is useful for game engines, physics simulations, etc. Where object relativity is needed for interactions. Entities and their positions can be stored inside data structures, where operations like insertion, deletion, movement, and querying can be performed.**



# Problem Statement

**As entity count and query workload increases, basic operations become slower when inappropriate data structures are used. So, which data structures can developers use to address scalability and efficiently manage objects and their positions in a two-dimensional environment?**

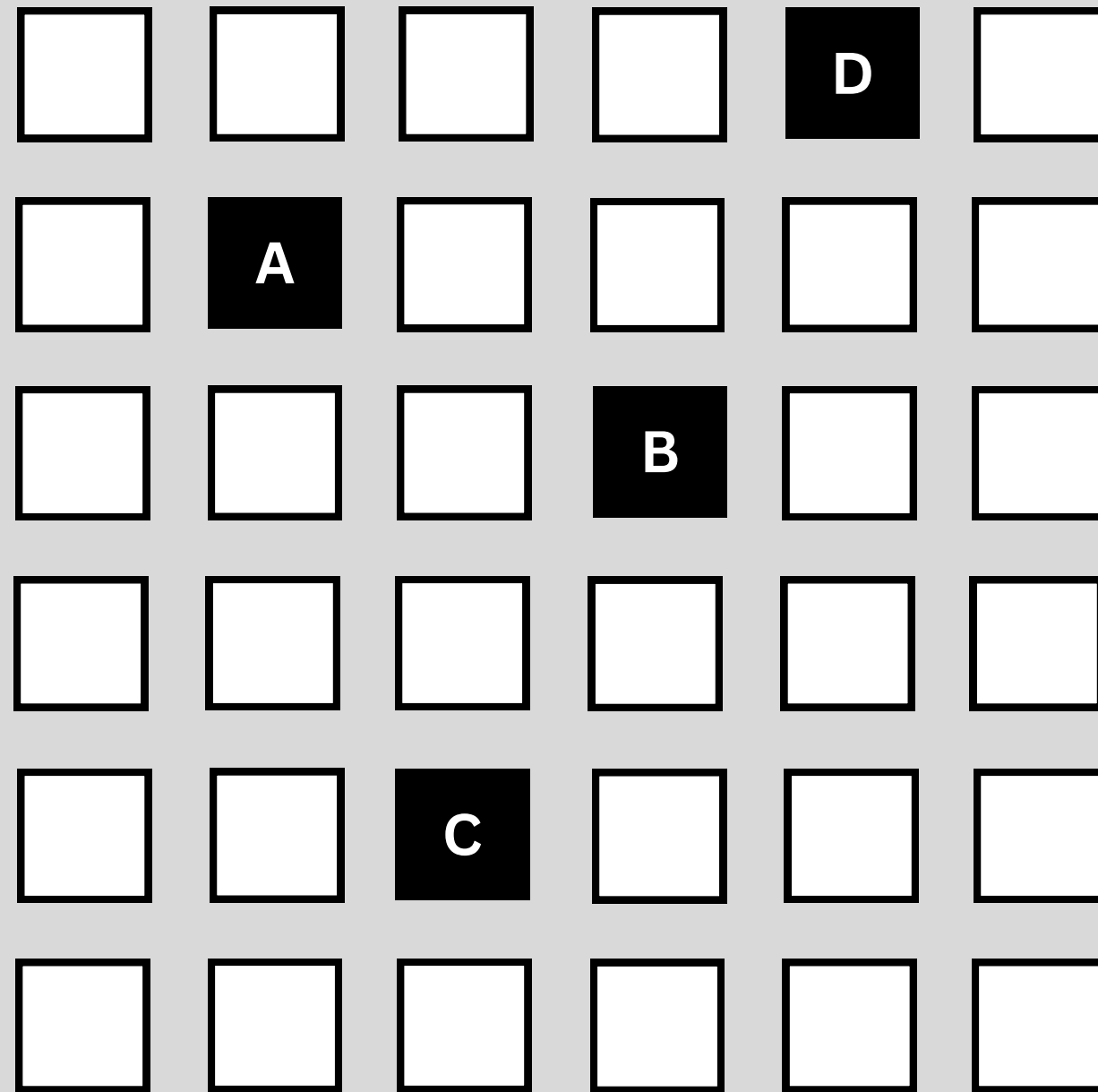




# Objectives

- **Implement multiple spatial data structures**
- **Compare runtime performance**
- **Analyze complexity**
- **Determine the most efficient structure**

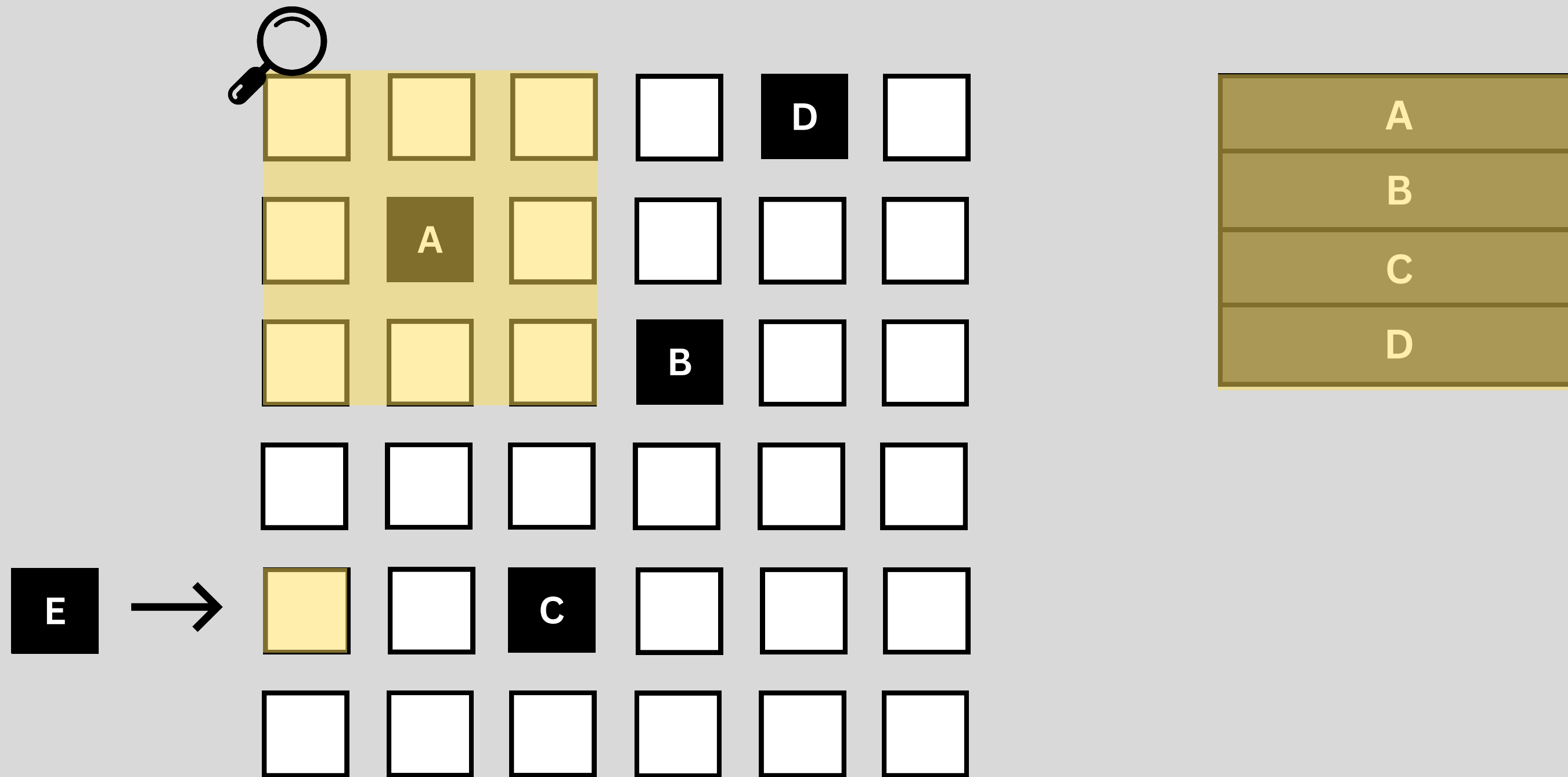
# Solution: LinkedList



|   |
|---|
| A |
| B |
| C |
| D |

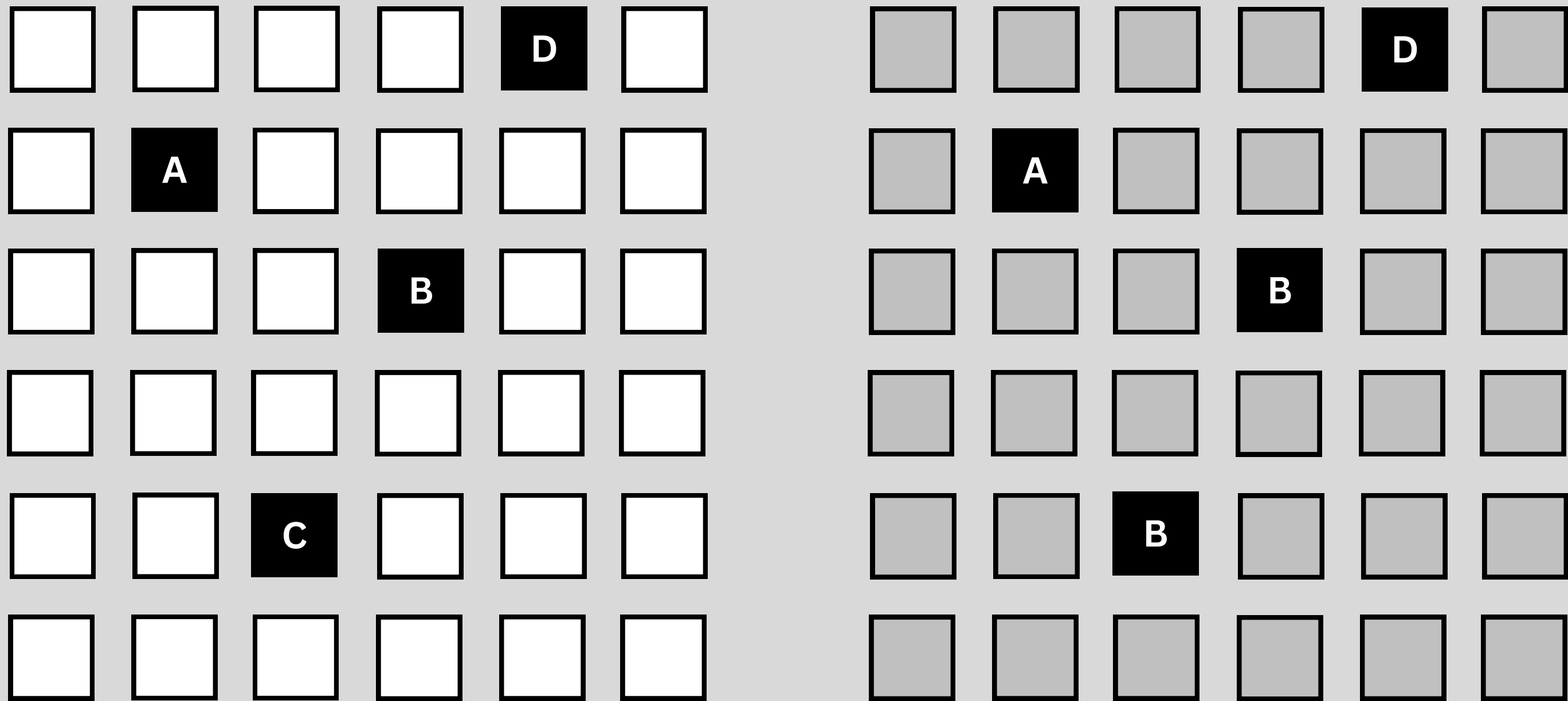
**Stores all objects linearly regardless of coordinates**

# LinkedList



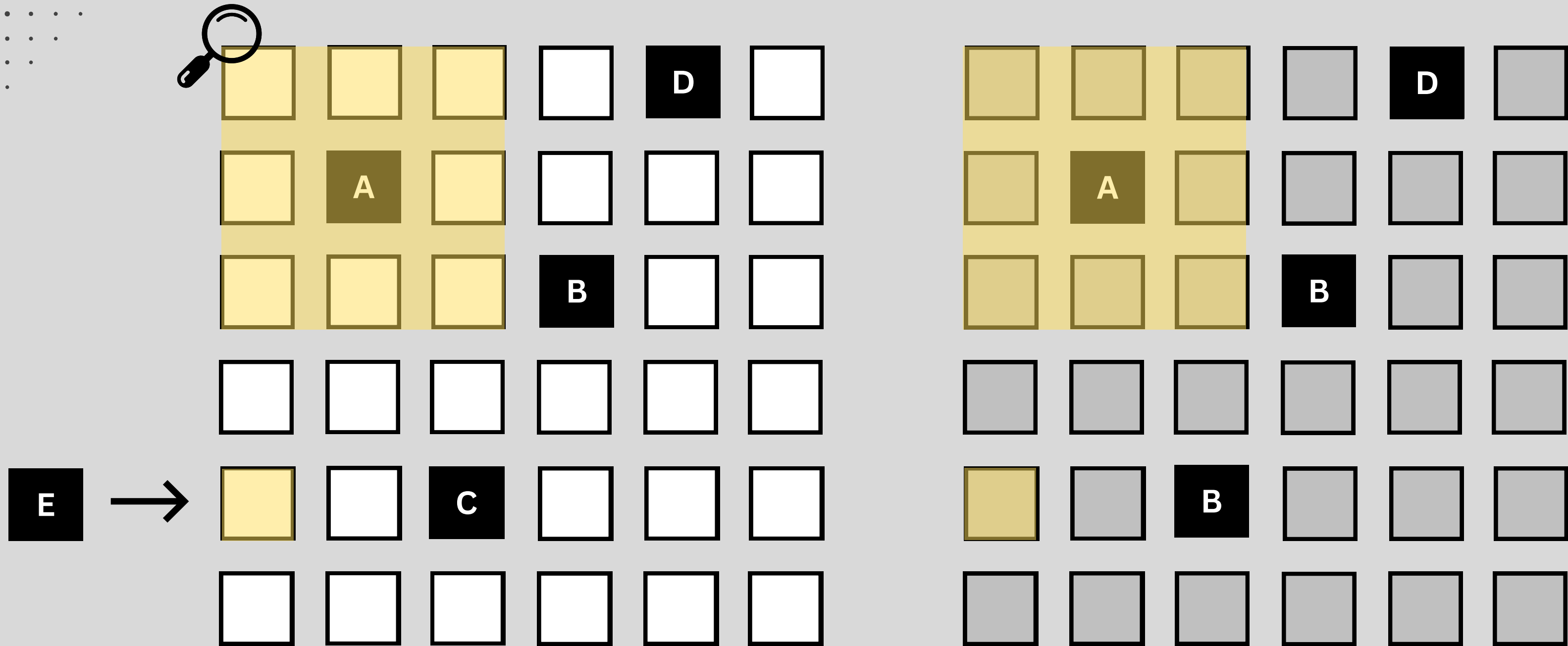
Scans everything just to insert or lookup an area

# Solution: 2D Grid



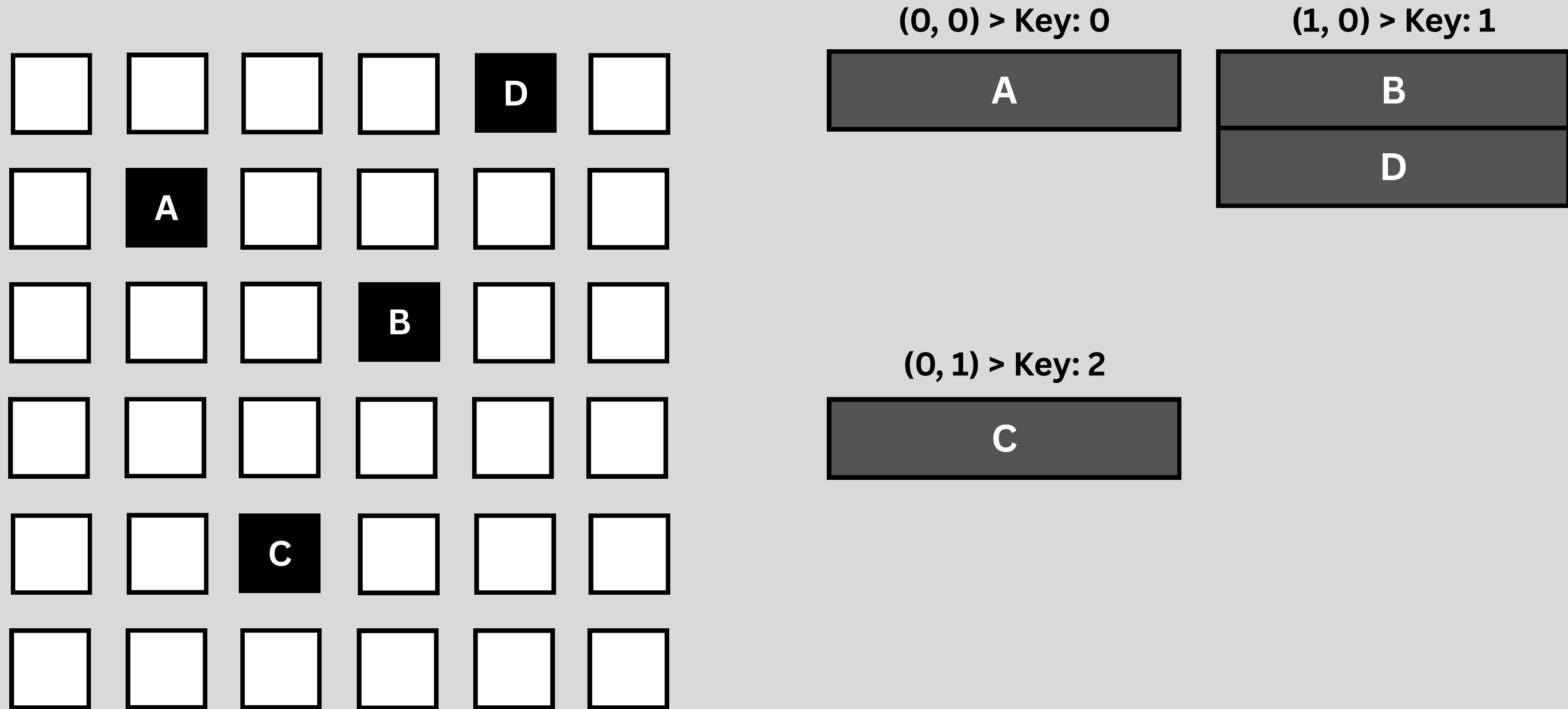
Creates a 2D array that mirrors the real map

# 2D Grid



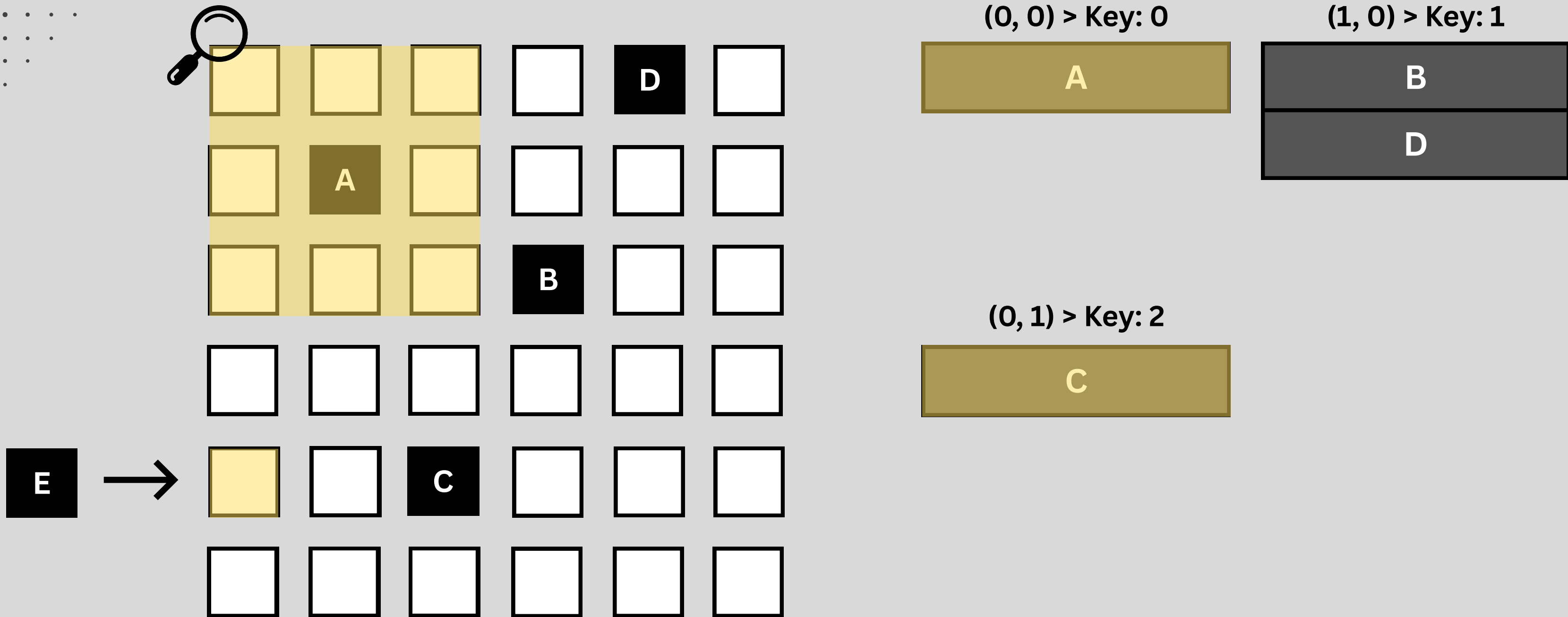
Scanning is direct and unaffected by global entity count, but is wasted on empty slots

# Solution: Hash Grid



Stores objects in separate LinkedLists, which  
are spatially indexed with hashing

# Hash Grid

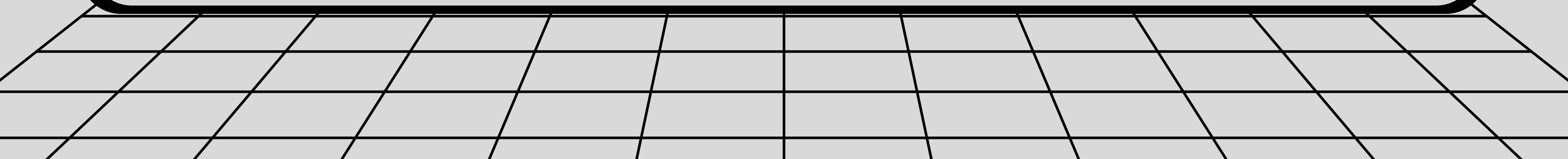


Scanning a point or area only scans local  
chunks, no empty slots are stored or scanned

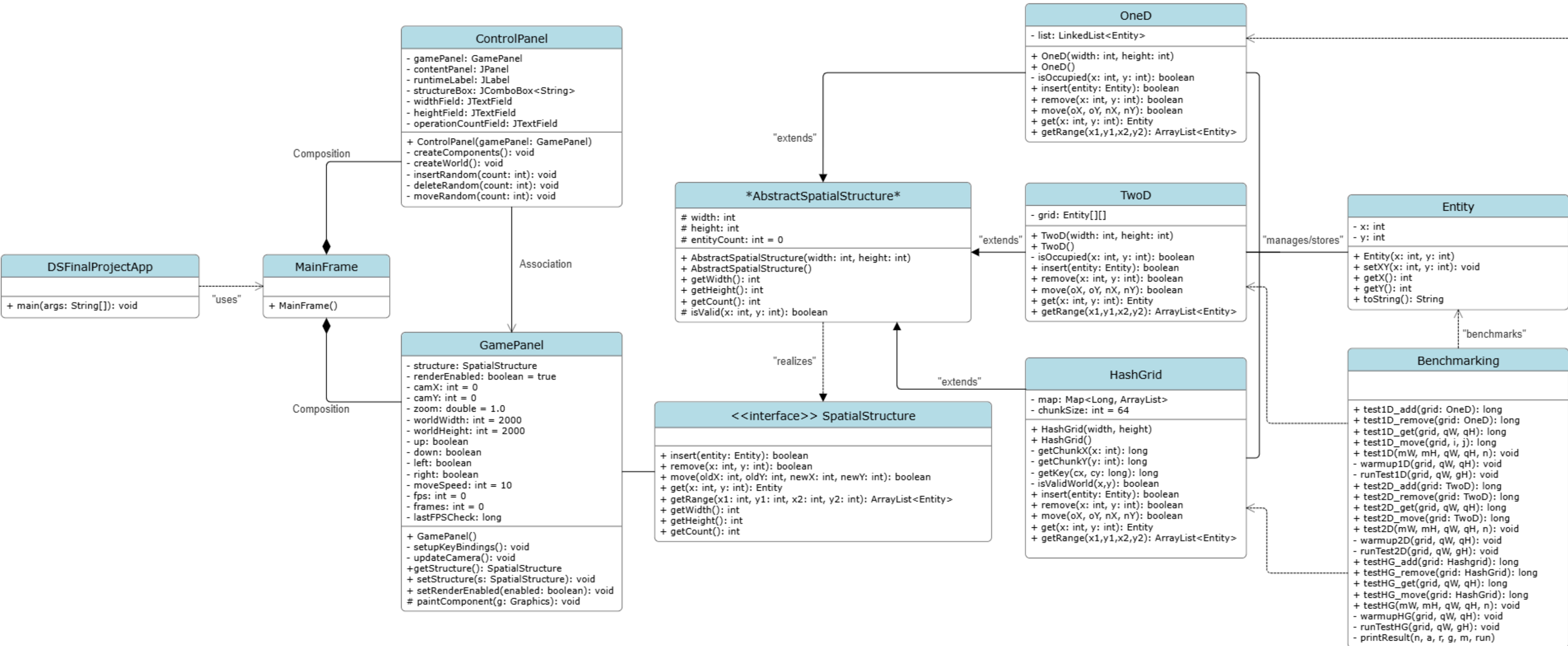


# Experiment Methodology

Each data structure will be evaluated by measuring the runtime performance in nanoseconds for insertion, deletion, position updates, and range query. Each operation are executed multiple times and averaged to reduce variance. Entity counts and query area sizes will vary between tests. Entity distribution and testing conditions will remain consistent for fairness, map size remains fixed at 2000 x 2000. Results will be compiled into tables and graphs for comparison and analysis.



# Implementation



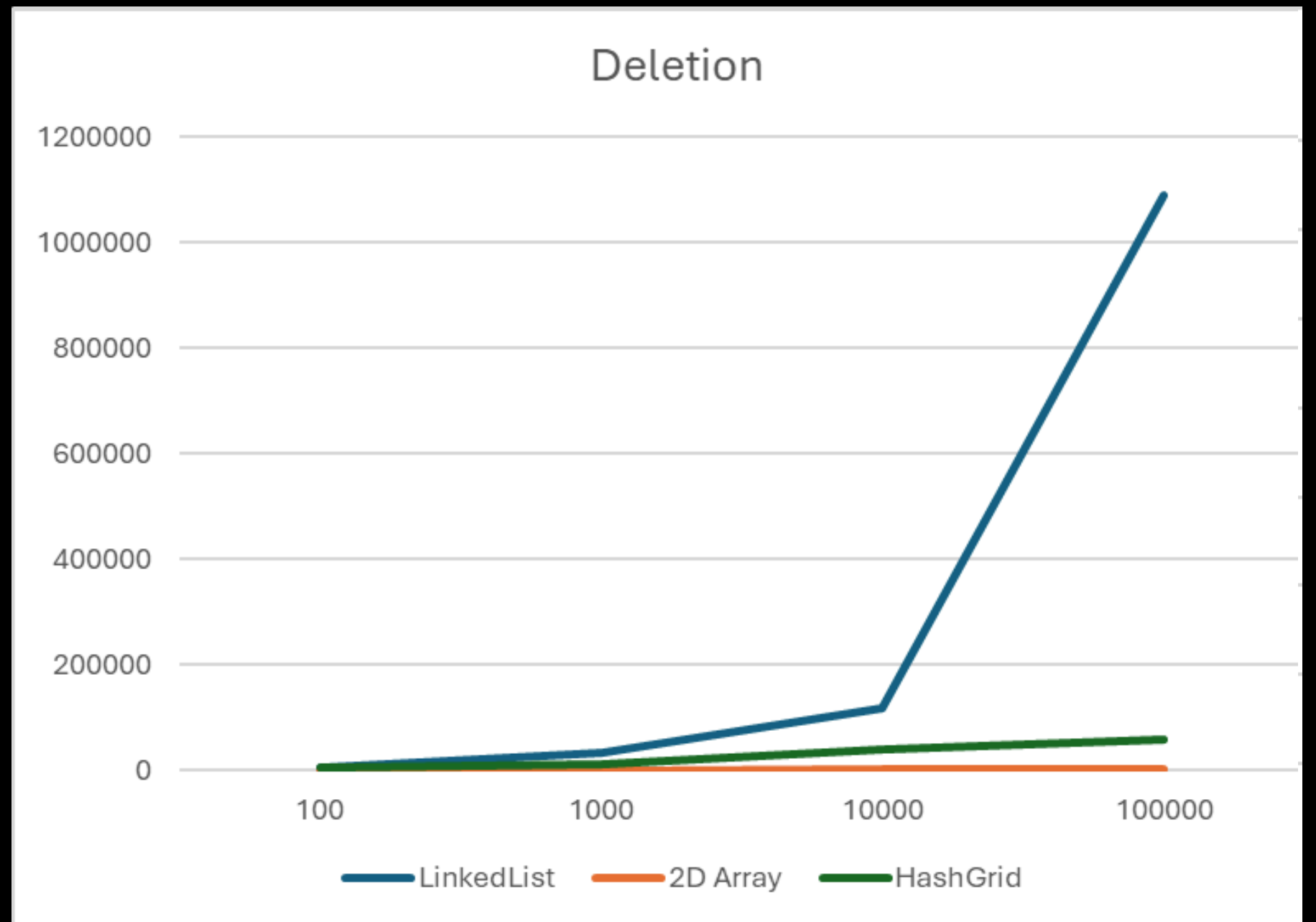
# Insertion

| Entity Count<br>(N) | LinkedList | 2D Array | HashGrid |
|---------------------|------------|----------|----------|
| 100                 | 2726       | 277      | 2664     |
| 1,000               | 8811       | 191      | 6694     |
| 10,000              | 54692      | 147      | 16792    |
| 100,000             | 624476     | 246      | 26667    |



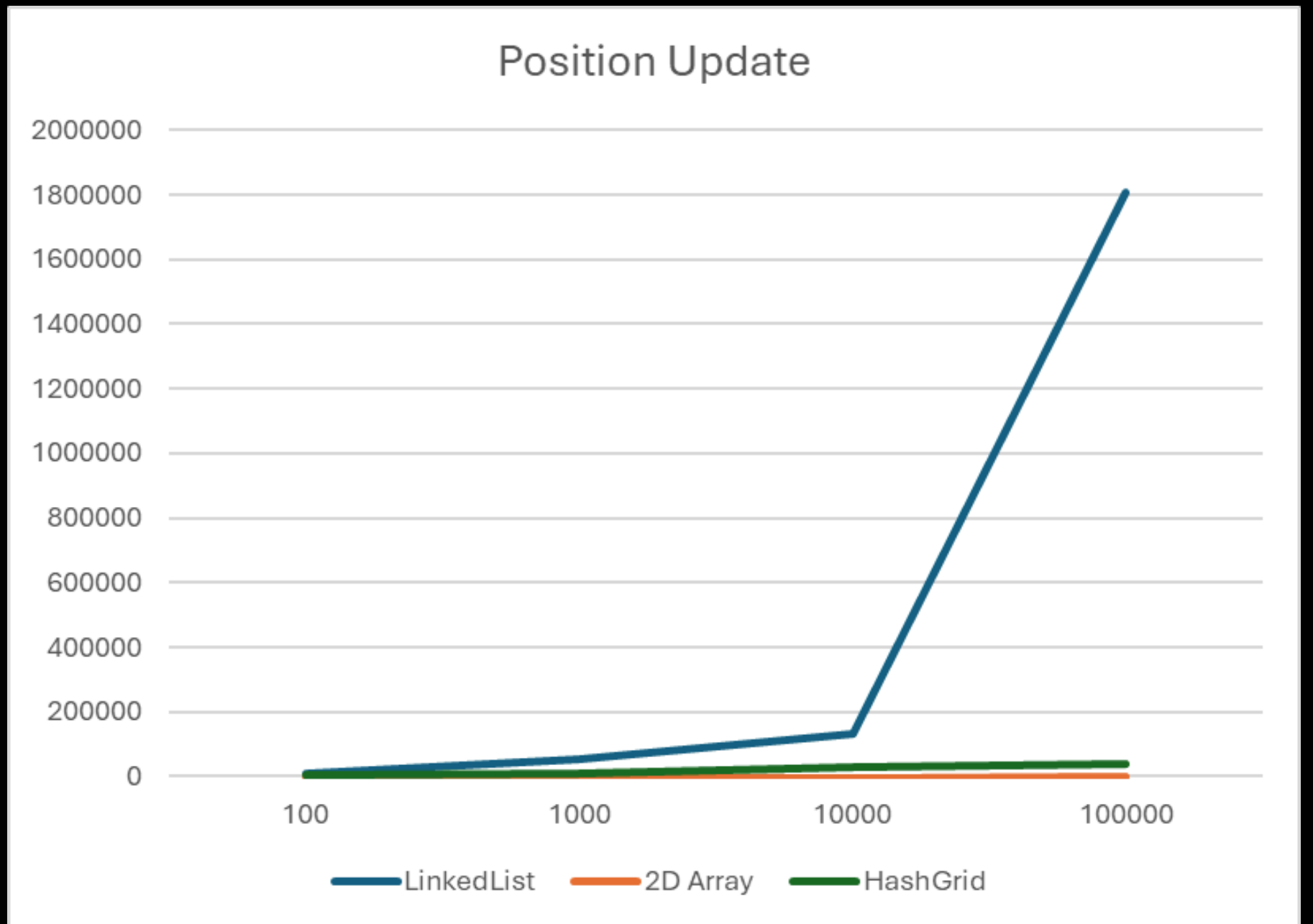
# Deletion

| Entity Count<br>(N) | LinkedList | 2D Array | HashGrid |
|---------------------|------------|----------|----------|
| 100                 | 4521       | 161      | 4693     |
| 1,000               | 33689      | 155      | 9880     |
| 10,000              | 118693     | 239      | 37929    |
| 100,000             | 1088810    | 273      | 59264    |



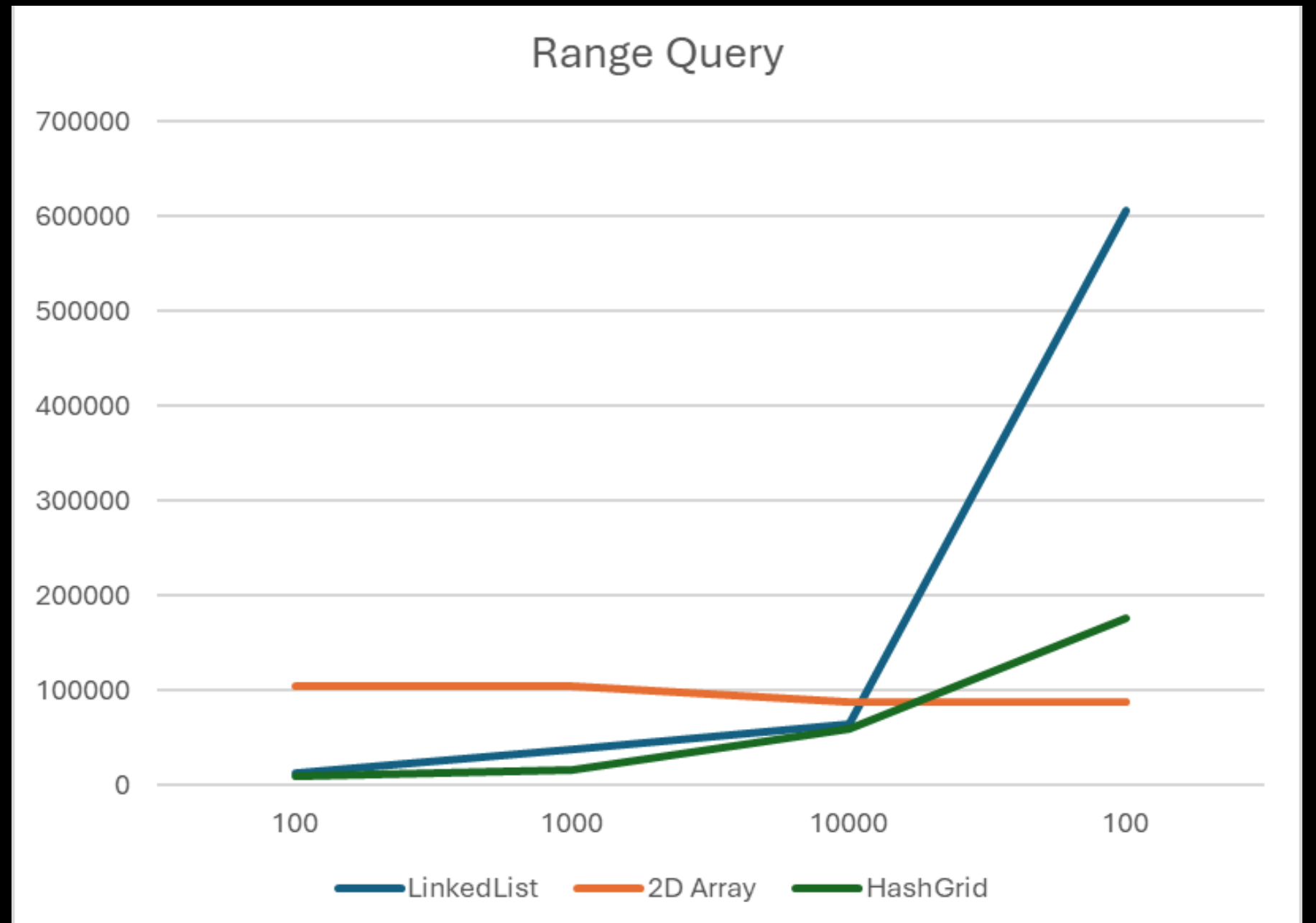
# Position Update

| Entity Count<br>(N) | LinkedList | 2D Array | HashGrid |
|---------------------|------------|----------|----------|
| 100                 | 8860       | 266      | 4913     |
| 1,000               | 51571      | 244      | 9378     |
| 10,000              | 131709     | 229      | 27929    |
| 100,000             | 1807648    | 276      | 39261    |



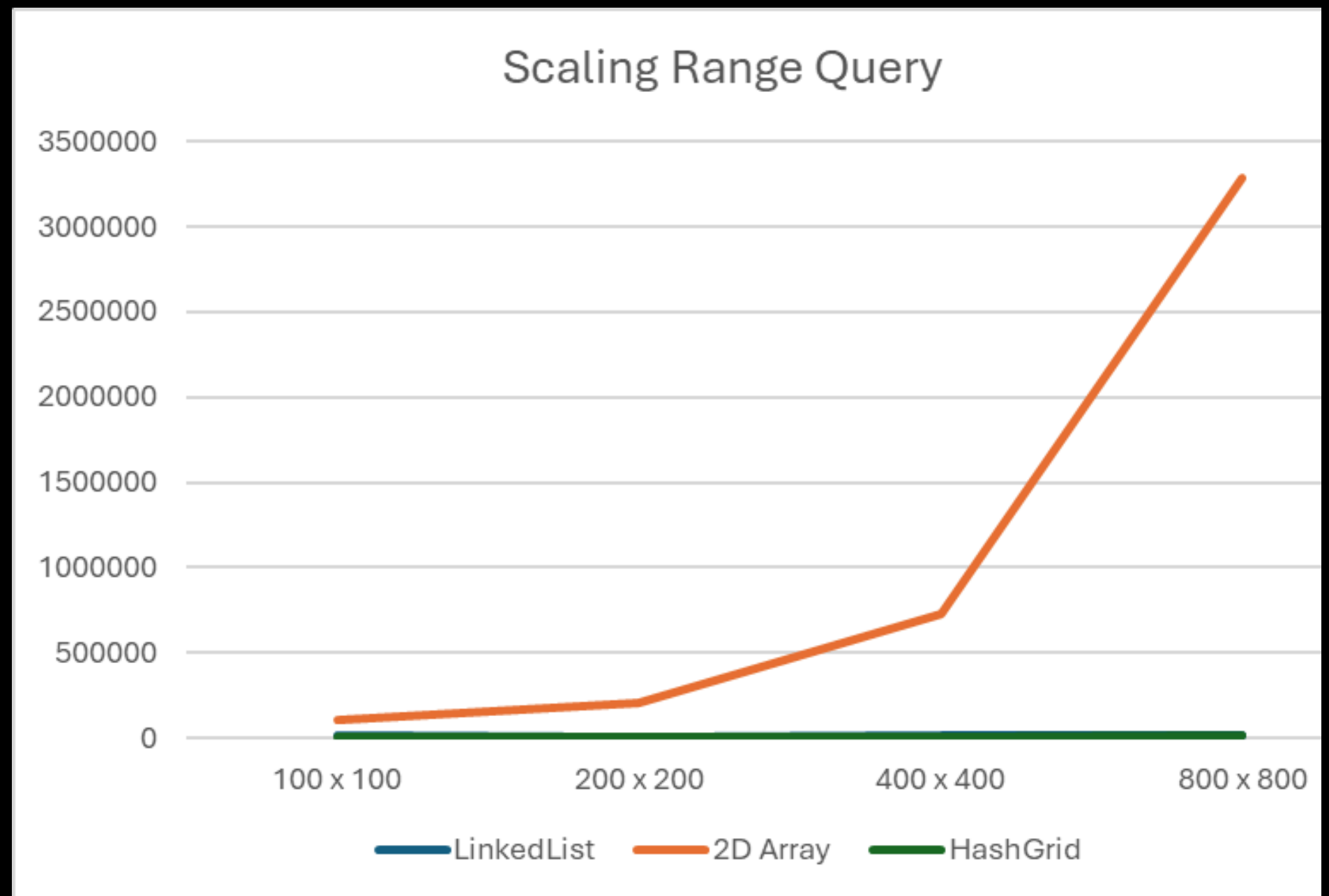
# Range Query

| Entity Count<br>(N) | LinkedList | 2D Array | HashGrid |
|---------------------|------------|----------|----------|
| 100                 | 11671      | 103655   | 9197     |
| 1,000               | 37309      | 103923   | 16571    |
| 10,000              | 63612      | 86865    | 59505    |
| 100,000             | 606308     | 88323    | 175903   |



# Scaling Range Query

| Query Area (Q) | LinkedList | 2D Array | HashGrid |
|----------------|------------|----------|----------|
| 100 x 100      | 11858      | 103908   | 9482     |
| 200 x 200      | 11050      | 203340   | 9696     |
| 400 x 400      | 11474      | 733080   | 10268    |
| 800 x 800      | 11299      | 3284139  | 11996    |



# Complexity Analysis

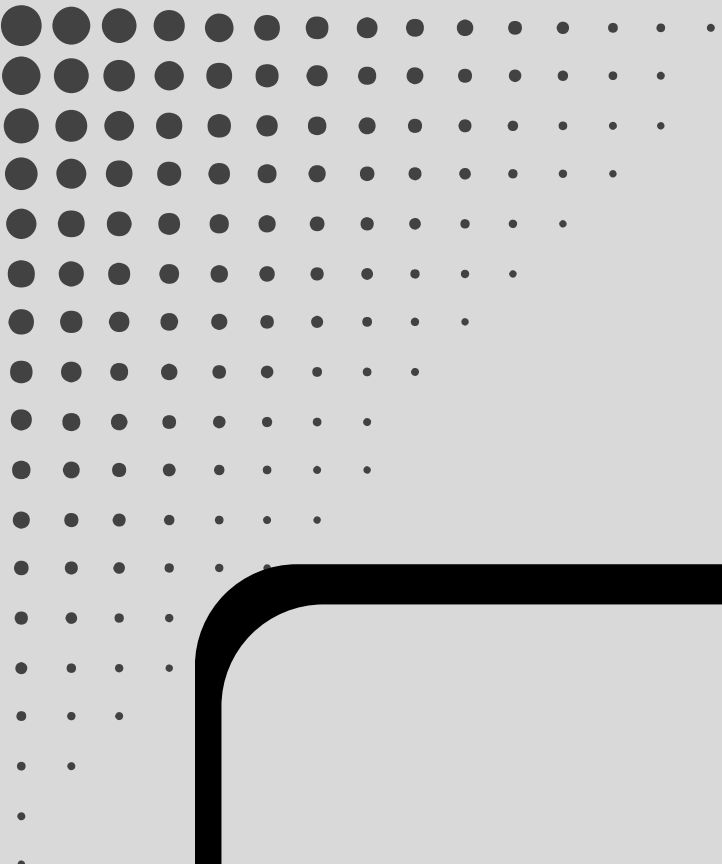
| Complexity Analysis | LinkedList | 2D Array | HashGrid                       |
|---------------------|------------|----------|--------------------------------|
| Insertion           | $O(n)$     | $O(1)$   | $O(c)$ or $O(1)$<br>Amortized  |
| Deletion            | $O(n)$     | $O(1)$   | $O(c)$ or $O(1)$<br>Amortized  |
| Position Updates    | $O(n)$     | $O(1)$   | $O(c)$ or $O(1)$<br>Amortized  |
| Range Query         | $O(n)$     | $O(q)$   | $O(kc)$ or $O(k)$<br>Amortized |

**$n$  = Entity count**

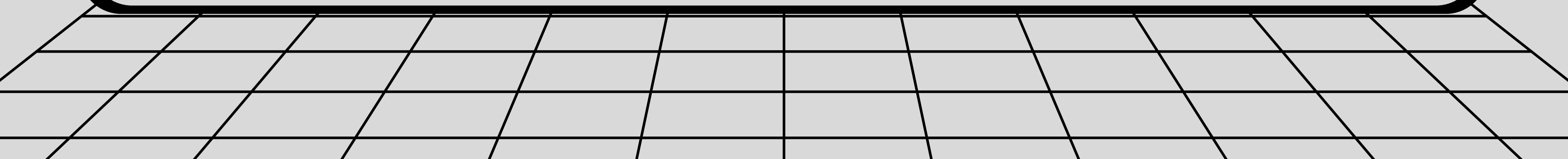
**$c$  = Chunk size**

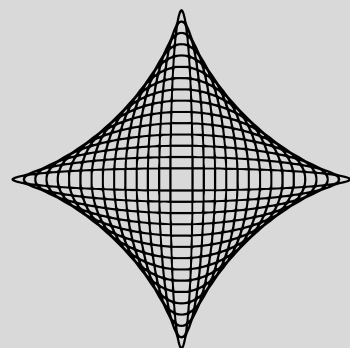
**$q$  = Coordinates queried**

**$k$  = Chunks queried**

A decorative pattern of dots in the top-left corner, arranged in a grid that tapers off to the right and bottom.

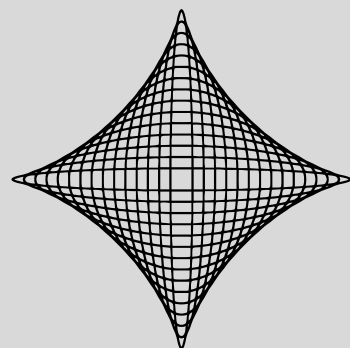
**App Demo!**

A perspective grid pattern at the bottom of the image, consisting of horizontal and vertical lines that create a sense of depth, resembling a floor or a stage.



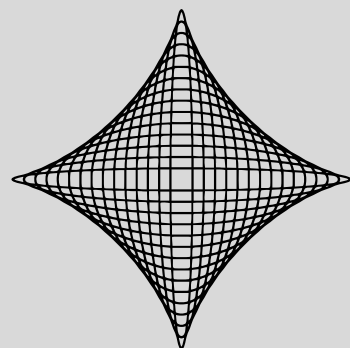
# Discussion

**LinkedList stores objects linearly, resulting in operations iterating through the entire structure to find entities with a given set of coordinates, lowering performance at high object count. Effective for lower entity count, but becomes inefficient at high object count.**



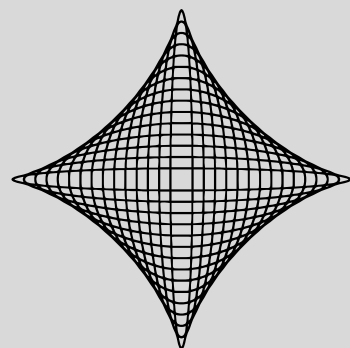
# Discussion

Two-dimensional arrays mirrors a two-dimensional space and represents coordinates with indexes, allowing for direct access to elements of a given set of coordinates, creating constant efficiency for update operations. But storing empty indexes increases memory cost and range query inefficiency.



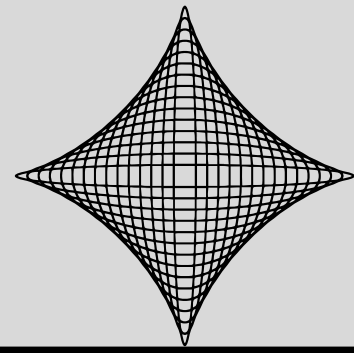
# Discussion

A Hash Grid stores spatially-indexed partitioned chunks of fixed sizes, increasing the efficiency of operations while limiting them to local chunks. All operations are primarily dictated by the amount of objects per chunk, which can be averaged to constant under uniform distribution, but is  $O(c)$  with high density. A hashmap also does not store empty addresses, although with hashing and bucket overhead.



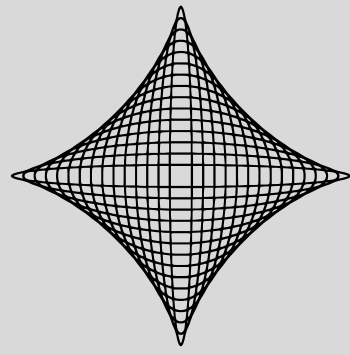
# Discussion

**For smaller programs with low entity count, linear data structures like LinkedList performs adequately. For dense environments, two-dimensional arrays provide efficient access and updates without wasting memory on empty slots due to density. For large scale projects with high entity count and frequent large queries, Hash Grids perform the best overall due to its fixed sized partitioning and spatial indexing.**



# Conclusion

**This project implements LinkedList, 2D arrays, and Hash Grids to a Data Structures benchmarking system using OOP principles. Each data structure was repeatedly measured and averaged in runtime on insertion, deletion, movement, and range queries, under scaling object count and query area size. Linear performs well with low object count, but scales linearly. 2D grids perform well with high object count, but wastes time and memory on empty slots. Hash Grids perform the best efficiency-wise for all operations with high scaling.**



# References

- Beckmann, N., Kriegel, H. P., Schneider, R., & Seeger, B. (1990). The R-tree: an efficient and robust access method for points and rectangles. ACM SIGMOD Record, 19(2), 322 - 331. [https://infolab.usc.edu/csci585/Spring2010/den\\_ar/p322-beckmann.pdf?utm\\_source](https://infolab.usc.edu/csci585/Spring2010/den_ar/p322-beckmann.pdf?utm_source)
- Bittner, J. & Wonka, P. (2003). Visibility in computer graphics. Environment and Planning B: Planning and Design, 30(5), 729 - 755. [https://journals.sagepub.com/doi/pdf/10.1068/b2957?utm\\_source](https://journals.sagepub.com/doi/pdf/10.1068/b2957?utm_source)
- Guttman, A. & Stonebraker, M. (1983). R-trees: a dynamic index structure for spatial searching. Proceedings of the ACM SIGMOD International Conference on Management of Data, 47 -57. [https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/Archive/ERL-m-83-64.pdf?utm\\_source](https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/Archive/ERL-m-83-64.pdf?utm_source)
- Hussain, S. M., Kumar, P. S. V. V. S. R., & Prasad, M. H. M. K. (2011). Performance evaluation of spatial indexing techniques. International Journal of Advanced Research in Computer Science, 2(5), 59 - 64. [https://www.researchgate.net/profile/Syed-Hussain-65/publication/260284221\\_Performance\\_Evaluation\\_Of\\_Spatial\\_Indexing\\_Techniques/links/54c4ba480cf2911c7a4f2a4d/Performance-Evaluation-Of-Spatial-Indexing-Techniques.pdf?utm\\_source](https://www.researchgate.net/profile/Syed-Hussain-65/publication/260284221_Performance_Evaluation_Of_Spatial_Indexing_Techniques/links/54c4ba480cf2911c7a4f2a4d/Performance-Evaluation-Of-Spatial-Indexing-Techniques.pdf?utm_source)
- Khot, P. G. & Srivas, S. (2018). Introduction of spatial database spatial data types, spatial indexing, geographic information systems, and query structure. Proceedings of International Conference on Advances in Computer Technology and Management, 6(3), 137 - 142. <https://docslib.org/doc/6051509/introduction-of-spatial-database-spatial-data-types-spatial-indexing-geographic-information-systems-gis-query-structure-dr>
- Mahmood, A. R., Aly, A. M., & Aref, W. G. (2017). Fast: frequency-aware spatio-textual indexing for in-memory continuous filter query processing. <https://arxiv.org/pdf/1709.02529>
- Pandey, V., Renen, A.V., Kipf, A., & Kemper, A. (2021). How good are modern spatial libraries? Data Science and Engineering, 6(2), 192 - 208. [https://link.springer.com/content/pdf/10.1007/s41019-020-00147-9.pdf?utm\\_source](https://link.springer.com/content/pdf/10.1007/s41019-020-00147-9.pdf?utm_source)
- Piah, P. Z., Asagba, P. O., Ejiofor, V., & Igulu, K. T. (2016). An efficient point data indexing structure for multidimensional range queries. International Journal of Engineering Research & Technology, 5(1), 699 - 703. [https://www.ijert.org/research/an-efficient-point-data-indexing-structure-for-multidimensional-range-queries-IJERTV5IS010586.pdf?utm\\_source](https://www.ijert.org/research/an-efficient-point-data-indexing-structure-for-multidimensional-range-queries-IJERTV5IS010586.pdf?utm_source)
- Piah, P. Z., Asagba, P. O., Ejiofor, V., & Igulu, K. T. (2016). Indexing structures for range searching with point objects: a survey. International Journal of Engineering Research & Technology, 5(1), 635 - 641. <https://www.ijert.org/research/indexing-structures-for-range-searching-with-point-objects-a-survey>
- Samet, H. (1984). The quadtree and related hierarchical data structures. ACM Computing Surveys, 16(2), 187 - 260. [https://www.cs.umd.edu/~hjs/pubs/SameCSUR84-ocr.pdf?utm\\_source](https://www.cs.umd.edu/~hjs/pubs/SameCSUR84-ocr.pdf?utm_source)

A decorative graphic in the top-left corner consisting of a grid of dots of varying sizes, arranged in a pattern that tapers off to the right.

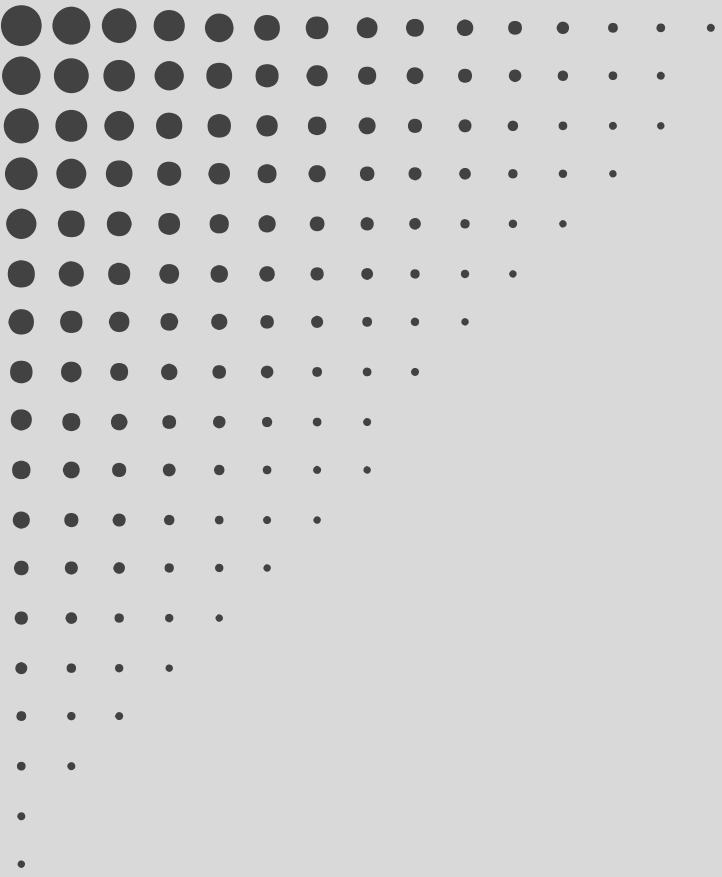
# Appendices

**Github Repository:**

**[https://github.com/AlmaTonara/DS\\_FP\\_Repo\\_123ssds32rdsax](https://github.com/AlmaTonara/DS_FP_Repo_123ssds32rdsax)**

**Poster, Repository, and Report Link:**

**[https://drive.google.com/drive/folders/1aR-z5lTZHv4lkzZia\\_UQ9en4jGcnV8vY](https://drive.google.com/drive/folders/1aR-z5lTZHv4lkzZia_UQ9en4jGcnV8vY)**



**Thank You**